

Telemedicine Platform

Progress & Technical Implementation Report

A three-role telehealth system — Patient · Doctor · Admin
Spring Boot 3.5 backend · Angular 21 frontend · Dockerised dev stack

Project	IRT42 — Telemedecine
Package root	com.irt42.telemedecine
Report date	3 June 2026
Status	Phases 0–2 complete · Phase 3 next
Context	Academic / school project

Contents

1. Executive Summary	3
2. Project Context	3
3. Progress Overview	4
4. Architecture at a Glance	4
5. Backend Implementation	5
5.1 Style: modular monolith	5
5.2 Modules built so far	5
5.3 REST surface	6
5.4 Persistence	6
6. Frontend Implementation	6
6.1 Angular 21, standalone & signals	6
6.2 Layout shells & UI primitives	6
6.3 HTTP interceptor chain	7
7. Authentication & Security	7
8. Doctor Verification Workflow	8
9. DevOps & Local Run	8
10. Simplifications & What's Next	8
10.1 Deliberate school-project simplifications	8
10.2 Immediate next step	9
Appendix A — Full REST Endpoint List	10
Appendix B — Database Tables	10

1. Executive Summary

This report documents the current state of a telemedicine platform built as an academic project. The application lets **patients** manage a personal health record and search a directory of verified doctors; lets **doctors** publish a professional profile and submit credentials for review; and gives **administrators** a verification workbench to approve or reject those doctors before they become visible to patients.

The system is delivered as a **modular-monolith** Spring Boot backend and an **Angular 21** single-page frontend, both packaged into a single-command Docker Compose stack. As of this report, three delivery phases are complete and verified by a full end-to-end walkthrough: a doctor signs up, uploads a diploma, an administrator reviews and approves it, and the doctor then appears in patient search under the correct specialty.

Project framing. This is a school project. The implementation deliberately favours **clarity of code** and **usability of the UI and backend logic** over production-grade infrastructure. Heavyweight cloud dependencies (S3/AWS, KMS, hardware-backed crypto) were intentionally dropped in favour of simple, local, self-contained equivalents that are easy to read, run and reason about. Where a production system would need a stronger mechanism, this is called out explicitly in Section 7 and Section 10.

72

Java source files

27

REST endpoints

19

Database tables

56

Angular TS files

3

Flyway migrations

5

Docker services

2. Project Context

The platform targets three distinct user spaces, each with its own navigation shell and permission boundary:

Role	What they do in the app today
Patient	Register, manage a medical record (allergies, treatments, lab results), and browse the directory of verified doctors filtered by specialty and language.
Doctor	Register, complete a public professional profile (title, bio, fee, specialties, spoken languages), and upload credential documents (diploma, board certification, license) for administrative review.
Admin	A seeded operator account that reviews pending verification cases, downloads and inspects each submitted credential, and approves or rejects the doctor — the approval flips the doctor to “verified” and makes them searchable.

The domain language and seed data are oriented to a Mauritanian context (currency shown as MRU/UM, French + English + Arabic among the spoken-language options), reflecting the original project specification.

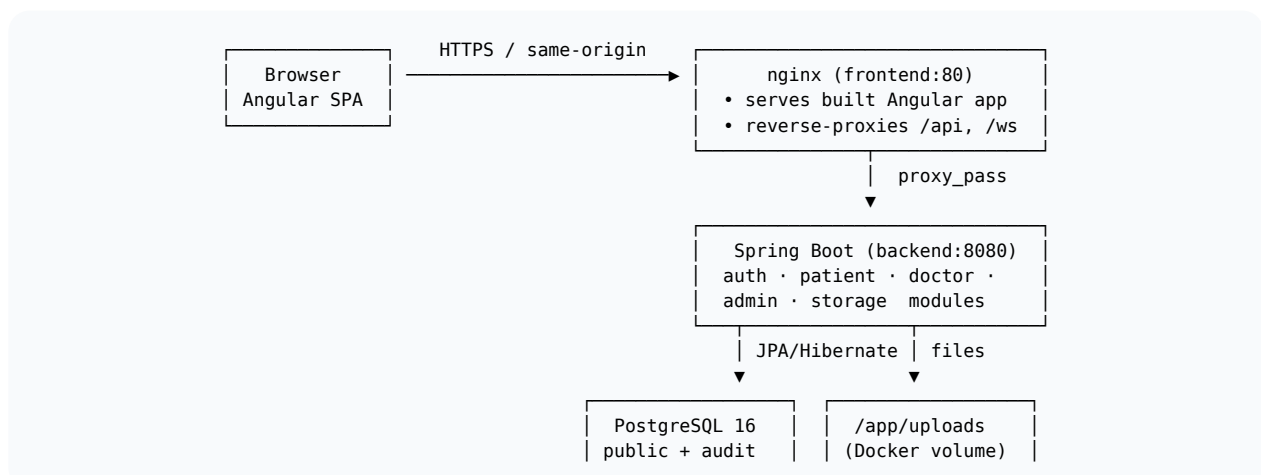
3. Progress Overview

Delivery is organised into seven phases (0–6). The foundational three are complete; the remaining four are designed and scoped in the project plan but not yet built.

Phase	Scope	Status
0 — Skeleton	Repo scaffolding, Spring Boot app, Flyway baseline, security config, full Angular route tree, design tokens, UI primitives, Docker stack.	DONE
1 — Auth & RBAC	Signup / login / refresh / logout, JWT issuance, TOTP two-factor, password hashing, refresh-token rotation, role-based guards and interceptors on the frontend.	DONE
2 — Profiles & Verification	Patient medical record CRUD, doctor profile + specialties + languages, credential upload, admin verification workflow, doctor search.	DONE
3 — Appointments	Doctor availability templates, slot computation, booking / reschedule / cancel, patient + doctor calendars.	PLANNED
4 — Consultation	LiveKit video rooms, WebSocket chat, shared files, doctor clinical notes, consultation report.	PLANNED
5 — Prescriptions & Payments	E-prescription PDF, simulated payment gateway, invoices, email reminders.	PLANNED
6 — Admin Analytics & GDPR	Versioned CMS, audit viewer, data export / deletion, reporting dashboards.	PLANNED

Dashboards are partially mocked. The patient and doctor dashboards render real layout and KPI components but are populated with representative sample data, clearly labelled in-UI (“real booking lands in Phase 3”, “mock payment data — Phase 5”). The live data wiring arrives with the phase that owns each feature.

4. Architecture at a Glance



Idle until later phases: MailHog (email) LiveKit (video)

The frontend never talks to the backend cross-origin: nginx serves the SPA and proxies `/api`, `/v3/api-docs`, `/swagger-ui`, `/actuator` and `/ws` to the backend on the internal Docker network, so the browser only ever sees one origin. This sidesteps CORS entirely and mirrors a realistic production topology.

5. Backend Implementation

5.1 Style: modular monolith

The backend is a single deployable Spring Boot application organised **by bounded context** rather than by technical layer. Each module owns a vertical slice of the domain and is internally split into the same four layers:

Layer	Responsibility
api	REST controllers + request/response DTOs (the HTTP edge).
application	Services holding the use-case logic and transaction boundaries.
domain	JPA entities and value objects — the persistent model.
infrastructure	Spring Data repositories and adapters (e.g. file storage).

Modules communicate across boundaries through Spring `ApplicationEvents` rather than direct calls, keeping contexts loosely coupled. For example, when a doctor uploads a credential the `doctor` module publishes a `CredentialUploadedEvent`; the `admin` module listens and opens or refreshes a pending verification case — neither module references the other's services directly.

5.2 Modules built so far

Module	Contents
common	Base entity (UUID id + timestamps), RFC 7807 error model, the global exception handler that maps exceptions to <code>problem+json</code> responses.
config	<code>SecurityConfig</code> (JWT resource server, BCrypt encoder, public-route allow-list), OpenAPI configuration.
auth	Accounts, roles, refresh tokens, TOTP secrets, password-reset tokens; the signup / login / refresh / logout / 2FA controllers and services; JWT minting.
patient	Patient profile and the medical-record sub-resources — allergies, treatments, lab results — with full CRUD under <code>/api/patients/me/*</code> .
doctor	Doctor profile, specialties (M:N), spoken languages, credential upload; the public doctor search and specialty list.
admin	Verification-case workflow: list, inspect, download credential, approve / reject; the dev-only bootstrap that seeds the first admin account.
storage	<code>LocalFileStorage</code> — writes uploads to the local filesystem under a per-area path and resolves them back safely (path-traversal guarded).

5.3 REST surface

The application currently exposes **27 endpoints** across five controllers. The full list is in Appendix A. A representative slice:

Method	Path	Purpose
POST	/api/auth/signup	Register a patient or doctor.
POST	/api/auth/login	Authenticate; returns tokens or a 2FA challenge.
GET	/api/patients/me	Fetch the caller's patient profile.
POST	/api/patients/me/allergies	Add an allergy to the record.
PUT	/api/doctors/me	Update the doctor's public profile.
POST	/api/doctors/me/credentials	Multipart upload of a credential file.
GET	/api/doctors	Search verified doctors (filterable).
POST	/api/admin/verifications/{id}/approve	Approve a doctor.

All reads are paginated where they return collections, all errors follow RFC 7807, and the contract is browsable live through the bundled Swagger UI at /swagger-ui.html.

5.4 Persistence

PostgreSQL 16 is the system of record, with schema evolution owned entirely by **Flyway**. Three migrations build the current 19-table schema:

Migration	Introduces
V1__init.sql	account, role (three seeded roles), account_role, refresh_token, email_otp, shedlock, and the append-only audit.audit_event in a separate schema.
V2__auth_2fa.sql	totp_secret, password_reset_token.
V3__profiles.sql	patient_profile, allergy, treatment, lab_result; specialty (ten seeded), doctor_profile, the doctor_specialty join, doctor_language, credential, and verification_case.

The schema is split into two PostgreSQL schemas: public for business data and audit for the append-only access log, the latter intended to be granted to an INSERT-only role so audit rows cannot be altered after the fact. Primary keys are UUIDs; open-in-view is disabled so persistence stays inside the service transaction boundary.

6. Frontend Implementation

6.1 Angular 21, standalone & signals

The frontend is a modern Angular 21 application built entirely from **standalone components** (no NgModules) with state held in **signals**. Routing is fully lazy and split by role; each role branch is protected by an authGuard plus a roleGuard so a patient token cannot reach /doctor/* or /admin/*.

6.2 Layout shells & UI primitives

Each role gets its own shell (patient-shell, doctor-shell, admin-shell) composed from a shared white sidebar with grouped navigation and a topbar with search, notifications and a user menu. Authentication screens use a separate two-column auth-shell with a gradient brand panel.

A small library of reusable primitives lives under shared/ui/ and is consumed across all features: Button, Input, Card, KpiCard (icon tile + trend pill + sparkline), StatusBadge, PageHeader, SidebarNav, Topbar, UserMenu,

EmptyState and an inline SVG Icon set. Design tokens — brand blue, neutrals, semantic status colours, radii and shadows — are defined once in a Tailwind 4 @theme block and consumed both as utility classes and as CSS variables, giving a single source of truth aligned to the Figma exports.

6.3 HTTP interceptor chain

Every request passes through four interceptors:

Interceptor	Role
authInterceptor	Attaches the bearer token; on a 401 it performs a single in-flight refresh (queuing concurrent requests behind a BehaviorSubject) and replays them, clearing the session only if the refresh itself fails.
errorInterceptor	Normalises transport errors and passes them through.
correlationIdInterceptor	Stamps each request with a correlation id.
loadingInterceptor	Drives a global loading indicator.

Public auth calls (/signup, /login, /refresh, /logout) are tagged with a SKIP_AUTH context token so the auth interceptor leaves them untouched. The session is persisted to localStorage and re-hydrated by parsing the JWT payload on load.

7. Authentication & Security

Concern	Implementation
Password storage	BCrypt at strength 12 (Spring Security BCryptPasswordEncoder).
Access tokens	Stateless JWT, HS256 (Nimbus MACSigner), 15-minute TTL. Claims: sub, roles, email, tfa_verified, iss, iat, exp, jti.
Refresh tokens	14-day TTL, stored hashed, rotated on every use ; a reused (stolen) token triggers a cascade-revoke of the family.
Two-factor	TOTP (RFC 6238) via dev.samstevens.totp, with a QR provisioning URI. Login returns a short-lived tfa_challenge token when 2FA is enabled; full tokens are issued only after the code verifies.
Authorisation	Spring Security 6 resource server maps the JWT roles claim to authorities; a public allow-list covers auth, health, docs and the specialty list — everything else requires a valid token.
Audit	A dedicated append-only audit.audit_event table and schema exist for PHI-access logging (wiring lands in a later phase).

School-project security posture. Several controls are deliberately simplified versus a regulated production system: passwords use BCrypt rather than Argon2id (which needed a heavyweight crypto dependency that was removed); medical data is stored in plaintext rather than field-encrypted; the JWT secret is a configured dev value. These are conscious trade-offs to keep the code readable and the stack self-contained — each has a clear upgrade path noted in the project's SECURITY.md.

8. Doctor Verification Workflow

The verification flow is the spine that ties all three roles together and is the project's fully working end-to-end demonstration:

1. **Sign up** — a doctor registers (`ROLE_DOCTOR`); admin self-registration is rejected.
2. **Build profile** — title, bio, consultation fee, specialties and spoken languages.
3. **Upload credential** — a single multipart POST stores the file via `LocalFileStorage` and publishes `CredentialUploadedEvent`.
4. **Case opens** — the `admin` module's listener opens (or refreshes) a `PENDING` verification case for that doctor.
5. **Admin reviews** — the operator lists pending cases, streams each credential document back through the browser, and decides.
6. **Approve** — the doctor's `verified` flag flips to true and credential statuses move `PENDING` → `APPROVED`.
7. **Discoverable** — the doctor now appears in patient search under their specialty.

This whole path has been exercised against the running Docker stack: a doctor uploaded a 4 MB PDF diploma, an admin downloaded and approved it, and a freshly registered patient found that doctor in a specialty search.

9. DevOps & Local Run

The entire system starts with a single command:

```
docker compose up -d --build
```

Service	Port	Role
frontend (nginx)	4200:80	Serves the SPA + reverse-proxies the API.
backend	8080	Spring Boot application.
postgres	5442:5432	Database (data on a named volume).
livekit	7880	Video server — idle until Phase 4.
mailhog	8025	Captured email — idle until later phases.

Both images use **multi-stage Docker builds** with BuildKit cache mounts (for `~/m2` and `~/npm`) to keep rebuilds fast. The nginx layer sets deliberate cache headers — `no-store` on `index.html` and the SPA fallback so a deploy never serves a stale shell, `immutable` on content-hashed assets — and raises `client_max_body_size` to accommodate credential uploads. Uploads persist to a Docker volume owned by the non-root `spring` user.

10. Simplifications & What's Next

10.1 Deliberate school-project simplifications

Production would use	This project uses
S3 / object storage with presigned URLs	<code>LocalFileStorage</code> on a Docker volume.
KMS + envelope encryption of PHI columns	Plaintext columns (audit table reserved).
Argon2id password hashing	BCrypt strength 12.

Real payment gateway (Stripe / local)	A mock gateway planned for Phase 5.
Managed email + push providers	MailHog captures mail locally.
Hosted, scaled LiveKit	Self-hosted LiveKit container for dev.

10.2 Immediate next step

Phase 3 — Appointments. Doctor availability templates and exceptions, server-side slot computation, and booking / reschedule / cancel, surfaced through a patient booking calendar and a doctor week-view agenda. The mock dashboards already reserve the space for this real data.

Appendix A — Full REST Endpoint List

Method	Path
POST	/api/auth/signup
POST	/api/auth/login
POST	/api/auth/refresh
POST	/api/auth/logout
POST	/api/auth/2fa/setup
POST	/api/auth/2fa/enable
POST	/api/auth/2fa/verify
POST	/api/auth/2fa/disable
GET	/api/patients/me
PUT	/api/patients/me
POST	/api/patients/me/allergies
DELETE	/api/patients/me/allergies/{id}
POST	/api/patients/me/treatments
DELETE	/api/patients/me/treatments/{id}
POST	/api/patients/me/lab-results
DELETE	/api/patients/me/lab-results/{id}
GET	/api/doctors
GET	/api/doctors/specialties
GET	/api/doctors/me
PUT	/api/doctors/me
POST	/api/doctors/me/credentials
GET	/api/admin/verifications
GET	/api/admin/verifications/{id}
GET	/api/admin/verifications/{id}/credentials/{credId}
POST	/api/admin/verifications/{id}/approve
POST	/api/admin/verifications/{id}/reject

Paths reconstructed from the controller mappings; consult the live Swagger UI at </swagger-ui.html> for the authoritative, parameter-level contract.

Appendix B — Database Tables

- account
- account_role
- role
- refresh_token
- email_otp
- totp_secret
- password_reset_token
- shedlock
- patient_profile
- allergy

- treatment
- lab_result
- specialty
- doctor_profile
- doctor_specialty
- doctor_language
- credential
- verification_case
- audit.audit_event